



Analysis and comparison of encryption and verification algorithms applied to a Fog Computing Architecture

Layla M. Martínez Guevara, Pablo Javier Vidal, Ana Carolina Olivera & Emmanuel N. Millán

To cite this article: Layla M. Martínez Guevara, Pablo Javier Vidal, Ana Carolina Olivera & Emmanuel N. Millán (2025) Analysis and comparison of encryption and verification algorithms applied to a Fog Computing Architecture, Journal of Information and Telecommunication, 9:2, 173-189, DOI: [10.1080/24751839.2024.2411884](https://doi.org/10.1080/24751839.2024.2411884)

To link to this article: <https://doi.org/10.1080/24751839.2024.2411884>



© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group



Published online: 08 Oct 2024.



Submit your article to this journal [↗](#)



Article views: 494



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 1 View citing articles [↗](#)



Analysis and comparison of encryption and verification algorithms applied to a Fog Computing Architecture

Layla M. Martínez Guevara^a, Pablo Javier Vidal ^{b,c}, Ana Carolina Olivera ^{b,c} and Emmanuel N. Millán ^{b,d}

^aUniversidad de Mendoza, Mendoza, Argentina; ^bInstituto Interdisciplinario de Ciencias Básicas ICB, CONICET-FCEN UNCuyo, Mendoza, Argentina; ^cFacultad de Ingeniería, Universidad Nacional de Cuyo, Mendoza, Argentina; ^dInstituto Tecnológico Universitario (ITU), Universidad Nacional de Cuyo, Mendoza, Argentina

ABSTRACT

Due to the Internet of Things (IoT) widespread adoption, data traffic to cloud servers for analysis has surged, impacting IoT system latency. Fog Computing emerged as a solution, bringing processing capabilities closer to data generation points, reducing data sent to the cloud, and improving response times. However, it intensifies security concerns due to the proximity of sensitive data to users and limited node computational resources. This research aims to study encryption and data verification algorithms using various datasets and the cryptography software OpenSSL. Testing is performed on Raspberry Pi hardware, facilitating the implementation of Fog nodes. Metrics such as CPU, memory, and energy consumption are evaluated using scripts. Results show that data type does not influence algorithm choice, and patterns related to data size behaviour are identified, aiding in defining security strategies.

ARTICLE HISTORY

Received 13 March 2024
Accepted 28 September 2024

KEYWORDS

Fog Computing; security; cryptography

1. Introduction

Fog Computing represents an innovative paradigm that broadens the cloud platform model by offering computing resources at the network's edge (Yi et al., 2015). It resembles a cloud platform with data, computing, storage, and application services, yet it distinguishes itself fundamentally through decentralization. Additionally, Fog nodes can process a larger amount of data locally than Internet of Things (IoT) devices operate on-site, are fully portable, and can be deployed on heterogeneous hardware. In essence, it brings some of the computing, data processing, and decision-making closer to the location where the data is generated (De Giusti et al., 2021). These characteristics make the Fog platform well-suited for time and location-sensitive applications, but they

CONTACT Emmanuel N. Millán  emmanuel.millan@itu.uncu.edu.ar  Instituto Interdisciplinario de Ciencias Básicas ICB, CONICET-FCEN UNCuyo, Padre Contreras 1300, Mendoza 5500, Argentina; Instituto Tecnológico Universitario (ITU), Universidad Nacional de Cuyo, Mendoza, Argentina

© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group
This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial License (<http://creativecommons.org/licenses/by-nc/4.0/>), which permits unrestricted non-commercial use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

also give rise to various challenges, including security issues related to data, virtualization, segregation, network, malware, and monitoring (Shruti et al., 2024).

The goal of Fog Computing is to shorten the communication paths between the cloud and devices and reduce the data flow in external networks. Fog nodes would thus play an intermediary role in the network, deciding which data is processed locally and which is sent to the cloud or data centre for analysis and processing (Qin et al., 2023).

Decentralized processing at the edge of the network also brings disadvantages, mainly arising from the work required to maintain and manage a distributed system. Fog Computing has its own limitations and unresolved challenges, such as Kahvazadeh (2019) and Perera et al. (2017): control and resource management, low-resource Fog devices, power limitation, connectivity, device heterogeneity, and security issues. Regarding this last challenge, Fog Computing, with its distributed nature and proximity to users, is more vulnerable than the cloud. Traditional security mechanisms for the cloud cannot be applied to the Fog system due to its distributed nature and lower computing power. Data security and privacy are significant challenges, both technically and due to legal and regulatory impediments (Khan et al., 2022).

There has always been concern about ensuring privacy in communications in IoT and Fog environments, trying to guarantee that a message could only be read by the recipient for which it was intended (Weber, 2015). There are various lightweight ciphers for IoT devices where computational resources are limited (Vishwanath et al., 2016). This challenge is even more difficult in the current world in which we live, where there is massive and intensive use of digital communications through the Internet. Cryptography plays a fundamental role in achieving this goal, helping us establish secure communications through the application of algorithms and security protocols. Concerning cryptography methods, three methods can be classified, among others: symmetric, asymmetric and hash function cryptography. Using various security measures and cryptographic algorithms at different locations consumes a lot of energy. So to reduce energy at various levels, appropriate cryptographic algorithms at appropriate locations need to be selected (Khan et al., 2022).

The use of a Fog model introduces significant data security and privacy challenges that must be addressed to ensure the safe and reliable operation of these systems. There are many issues in order to keep acceptable security at different levels. First, the heterogeneity between devices and platforms in a Fog environment complicates the implementation of uniform security policies. Different devices may support varying levels of security protocols depending on processing capacity and other hardware characteristics, making it difficult to ensure consistent protection across the network. With respect to data, analyse data without compromising individual privacy poses a significant challenge. Also, establishing trust among various Fog nodes and between nodes and users is essential for maintaining privacy. Nodes must be able to authenticate each other and ensure that data exchanges occur within a trusted framework. Given all of the above, it is necessary to establish a detailed analysis of how various security algorithms behave in heterogeneous environments and at the same time how their operation directly affects the performance of Fog devices.

In this work, the focus is on testing different cryptography algorithms within the three classifications mentioned above in a Fog computing environment. The primary goals involve the implementation of diverse data encryption algorithms within a Fog

Computing architecture, whilst the parameters such as key size, types of data, and sizes to be encrypted are varied. The aim is to assess performance metrics using different hardware configurations. The studied metrics focus on analysing the performance of the algorithms from a hardware point of view. In devices with limited computing capabilities, it is necessary to know how each algorithm affects the general performance of the device. The metrics analysed are: CPU consumption, Input Output Wait (IOWait), main memory usage, external memory accesses, energy consumption and execution time. To measure these metrics we use various software available in the GNU/Linux operating system such as *perf*, *sar* and *strace*. Additionally, the focus extends to the study and evaluation of checksum algorithms, examining their performance when combined with the implemented encryption techniques. This allows an evaluation of the trade-off between security and performance, where not only the computational performance is important, but also the encryption techniques chosen.

The evaluation will be conducted on actual hardware under isolated conditions, ensuring there is no system overload and only system software running on the Fog nodes. Data encryption and decryption tests will be executed using datasets sourced from IoT devices on the Fog node.

To accomplish these objectives, several scripts in the Bash language are developed, and open-source software for data encryption/decryption and checksum operations is used. These scripts are executed on three Raspberry Pi models. Through systematic evaluations of each execution, different algorithms to determine the combination that minimizes resource consumption on a Fog node are compared.

1.1. Related work

Considering the limitations of resources in Fog Computing, a lightweight cryptography technique is preferred. So, the Fog nodes are responsible for preventing the propagation of malicious packets over the cloud. Exist in the literature several works related to cryptography in Fog Computing.

Vishwanath et al. (2016) implement a process of encrypting over three different dataset types to prove how to Fog layer works using the Advanced Encryption Standard (AES) algorithm. The system is based on the encryption of the encrypted information. EAS has a fixed block size of 128 bits and key size of 128, 192 or 256 bits. AES operate on the 4x4 column-major order. The number of repetitions for each the size of bit key was set to 10 cycles of repetition for 128 bit key, 12 cycles of repetition for 192 bit key, and 14 cycles of repetition for 256 bit key. The metrics used for evaluation were user load vs CPU time, file size vs encryption time, file size vs decryption time and file size vs memory utilization. The results showed that datasets are encrypted and decrypted correctly. A cellphone with Android was used as an edge device, where the encrypted data were taken. The time of encrypted and decrypted, the user load, response time, and memory use over the file were evaluated. Based on the analysis, the authors conclude that AES is the safer encryption algorithm.

In the work of Patel and Bhavsar (2021) they change the size of the key and entry implementing security with the AES algorithm. The number of rounds for processing the plain text is increased to 22, where the first 21 rounds perform all four operations of the AES algorithm, and the final round implements three of the four operations for

encryption. Decryption was done by using the inverse operations of the encryption algorithm. AES-S will be suitable for applications with high security and throughput requirements without increasing the overall design when compared to the original AES-128 bits.

Several works are appointed to compare methods to select the best technology and encrypt algorithm. Alshambri and Alassery (2021) provide a comparison among RSA, AES, and ECC algorithms for Fog-enabled cybersecurity systems. These Algorithms are compared by developing them using Python-based language program, in terms of encryption/decryption time, key generation techniques, and other features offered. In addition, they proposed to use a hybrid cryptography system of two types of encryption algorithms such as RSA with AES to fulfil the security, file size, and latency required for the communication between the Fog and an e-learning platform for education. The authors combine encryption algorithms to provide less processing time and better response than cloud systems. A security analysis is performed and is validated using a test bed with Python, ASP.net and WCF services. The results demonstrate the necessity of encryption to prevent attacks and vulnerabilities.

In Froiz-Míguez et al. (2018) the authors analyse the effect of energy consumption in a house IoT system using the encryption to ZWiWi communication with a local gateway. For NodeMCU-based modules, TLSv1/SSLv3 with ECDHE-RSA-AES128-GCM-SHA256 was used as the cipher suite, while for ZigBee-based nodes, Application Support Sublayer encryption was enabled (APS). Measurements were obtained using an Arduino and an Adafruit INA219. The use of a Secure Sockets Layer (SSL) generates additional computational and communications load that increases energy consumption. However, employing a lighter (and less robust) encryption mechanism such as ZigBee APS barely increases average consumption but enhances privacy and security compared to not using any kind of encryption on the nodes.

Ramzan Ali et al. (2020) compare the efficacy of transferring Patient Healthcare Information with only two different encryption standards, AES and Rivest-Shamir-Adleman (RSA). The algorithms are codified inside Raspberry Pi hardware using open-source Python software. The dataset used in the work is the Coronary Artery Disease (CAD) which includes 54 features of 303 patients. The information is in a .csv file and can be directly accessed by the Fog computing device. The authors conclude that AES was superior to RSA regarding memory use and time required for encryption.

Blockchain and lightweight cryptography have been used recently for healthcare systems. In Janardhana-Naidu and Ganesh (2023) they introduce a lightweight encryption for healthcare data. The authors proposed a blockchain with Tiny Lightweight Symmetric Encryption which chooses the best key considering the Aquila Optimization Algorithm. The method is implemented in Python and compared with Rivest-Shamir-Adleman (RSA) and elliptical curve cryptography (ECC). For evaluation, the authors use a laptop and healthcare data is retrieved of the UCI machine library. Another example of blockchain and Fog computing in healthcare systems is introduced by Cheikhrouhou et al. (2023). The authors introduce a lightweight Blockchain-based and Fog-enabled remote patient monitoring system that provides a high level of security and efficient response time. The augmentation of Fog computing improved the responsiveness of the remote patient monitoring system by 40%. However, the infrastructure is expensive and needs a high initial investment.

When analysing different algorithms for Fog Computing environments, scalability performance analysis is crucial due to the dynamic and distributed nature of these systems. In general, many studies do not evaluate performance as the size of the data to be transmitted increases. Algorithms must maintain high performance even as the number of connected devices and the volume of data increases. In terms of computational demands of encryption algorithms can impact the performance of Fog nodes, which often have limited processing power. Algorithms with lower computational overhead are preferable to ensure that nodes can efficiently handle encryption tasks alongside other operations. The works do not evaluate the performance of simple to complex algorithms, but in most works they choose to seek to evaluate algorithms with low computational cost, avoiding evaluating those with high computational cost.

Scalability in Fog Computing is multifaceted, encompassing performance, resource utilization, energy efficiency, latency, network efficiency, interoperability, security, and adaptability as main characteristics to take into account. When evaluating encryption algorithms in Fog environments, these characteristics ensure that the chosen configuration can handle the unique challenges posed by the distributed and dynamic nature of Fog networks. By addressing these scalability factors, we can select robust and efficient encryption solutions that enhance the security and performance of Fog Computing environments.

In this work, the objective is to study encryption and data verification algorithms with the cryptography software OpenSSL in different contexts like video, image and text over a limited infrastructure such as Raspberry Pi.

The manuscript is organized as follows: Section 2 describes the cryptography algorithms, the hardware and software and the data sets of Materials and Methods. In Section 3 it is shown the results obtained along with a discussion, and finally in Section 4 it is detailed the conclusions and possible future work.

2. Materials and methods

In this section, the symmetric, asymmetric, and hash cryptography algorithms employed, outline the hardware and software utilized for conducting the tests, detail the metrics considered, and provide an overview of the datasets are presented.

2.1. Cryptography algorithms

In this study, a range of cryptography algorithms, including symmetric, asymmetric, and hash function methods are employed.

Symmetric cryptography encompasses a suite of algorithms and software implementations facilitating the encryption and decryption of information using a single encryption key (Abood & Guirguis, 2018; Mitali & Sharma, 2014). Notable algorithms within this category include DES, 3DES, AES, IDEA, Blowfish, and Chacha20.

In contrast, asymmetric cryptography utilizes two keys, a public key and a private key, earning it the designation of 'Public Key Infrastructure' (PKI), in conjunction with asymmetric encryption and authentication mechanisms. This approach is based on a key pair, ensuring the existence of only one public key for a corresponding private key, and vice versa. Additionally, a crucial condition for key generation is that the private key

cannot be derived from the public key. This condition is essential for any asymmetric algorithm, particularly considering the exchange of public keys between encryption nodes over potentially hostile mediums like the Internet. Well-known algorithms falling under this category include RSA, DSA, and Elliptic Curve cryptography (Shen et al., 2021).

A hash function is an algorithm that transforms an arbitrary set of data elements, such as a text file, into a single fixed-length value (the 'hash') (Chi & Zhu, 2017). The calculated hash value can be used to verify the integrity of copies of original data without the need to provide the original data. This irreversibility means that a hash value can be freely distributed or stored, as it is only used for comparison purposes.

The effectiveness of a hash algorithm in providing security relies heavily on its capacity to generate a unique value for a given data set. When a hash function yields the same value for two distinct data sets, a collision is deemed to have occurred. Such collisions elevate the risk that an attacker can strategically manipulate data sets to gain access to secure information or modify computer data files in a manner that preserves the hash value, thus avoiding detection. A robust hash function exhibits resistance to these computational attacks, whereas a vulnerable hash function is characterized by a high likelihood of collision occurrence. Lastly, a compromised hash function is one for which known computational methods exist to deliberately induce collisions.

For the experiments in this work, they are selected the following algorithms:

- Symmetric algorithms:
 - AES-256-CBC (block cipher) (Dworkin, 2001)
 - 3DES (block cipher) (The National Institute of Standards & Technology (NIST), 2017)
 - CHACHA20 (stream cipher) (Bernstein, 2008)
- Asymmetric algorithms:
 - RSA (Rivest et al., 1978)
 - Elliptic Curves (X25519) (ECDH) (Bernstein, 2006)
 - Elliptic Curves (secp256k1) (ECDSA) (Johnson et al., 2001)
- Hash functions:
 - SHA-1 (Wang & Yu, 2005)
 - SHA-512 (Dworkin, 2015)
 - SHA3-512 (Bertoni et al., 2013)
 - Ripemd-160 (Dobbertin, 1998)

2.2. Hardware

Three models of Raspberry Pi to emulate Fog nodes, intending to compare the execution times of the algorithms and assess the utilization of various resources are employed. Raspberry Pi is a small-sized and low-cost computer developed in the United Kingdom by the Raspberry Pi Foundation. The main objective of these devices is stimulating the teaching of computer science to children in their schools and to individuals (Raspberry Pi, n.d.; Zhao et al., 2015). See Table 1 for the hardware specifications of the three used models.

The Raspberry Pi is a small computer that needs an operating system to start all the necessary processes so that the hardware and computing capabilities are available for the applications you want to run. Since it does not have an internal storage unit, that operating system must be recorded on a micro-SD card. The operating system chosen

Table 1. Hardware specifications of the three Raspberry Pi models used in this work.

Features / Model	RPI 1 Model B+	RPI 3 Model B	RPI 4
CPU	ARM1176JZF-S 700 MHz	ARMv8 64-bit Quad-Core 1.2 GHz	ARM Cortex-A72 1.5 GHz
GPU	VideoCore IV	VideoCore IV	VideoCore VI
RAM memory	512 MB	1 GB	1/2/4 GB LPDDR4
USB	4 USB 2.0	4 USB 2.0	2× USB 2.0 2× USB 3.0
Ethernet	yes	yes	yes
Video output	HDMI, RCA	HDMI, RCA	HDMI, Micro HDMI
Audio output	3.5 mm, HDMI	3.5 mm, HDMI	3.5 mm, Micro HDMI
Storage	Micro SD	Micro SD	Micro SD
Size	85.6 × 56 mm	85.6 × 56 mm	88 × 58 mm
Energy consumption (A)			
Boot (Max)	0.26	0.75	0.85
Boot (Avg)	0.22	0.35	0.7
Idle	0.20	0.30	0.6
Stress	0.35	1.34	1.25

for this project is Debian, whose version for Raspberry Pi is known as Raspbian 11.5, it is the official and most used one and can be downloaded for free.

2.3. Software

In this section, scripts development for this work are introduced. The scripts are implemented in the Bash language to perform all the tests. The scripts use performance analysis tools and allow you to obtain metrics on different aspects of hardware and software used on Linux systems. It is used *perf* to obtain the values of various hardware counter metrics present in the CPU and in the Linux system (such as external memory accesses, cache misses, L1 data cache loads, etc). The *sar* tool allows to obtain, among other values, the time that the CPU spends *idle*, is *used* or in *IOWait*. The *strace* tool is utilized to obtain the number of write and read system calls.

The scripts were written in Bash language (version 5.1.4) and the encryption, decryption and checksum functions were used from the OpenSSL library version 1.1.1n. The scripts have the following structure:

- (1) Generate a private key.
 - (a) To encrypt using the asymmetric algorithm, create a public key and a private key. First, generate the private key with the selected asymmetric algorithm.
- (2) Generate a public key
 - (a) The public key is generated from the private key generated in point 1.
- (3) Encryption
 - (a) Encrypt with a key.
 - (b) Encrypt a file from the dataset using a symmetric algorithm and a password with the 'openssl enc' function.
 - (c) Repeat encryption to obtain metrics using the tools *ps*, *perf*, and *strace* included in the Raspbian Linux distribution.
 - (d) Extract the selected metrics and save them in a file.
- (4) Checksum and HMAC

- (a) Apply the digest function and sign or HMAC. A message digest algorithm takes a single input (a message) and produces a 'message digest' or hash that allows you to verify the integrity of the message. This hash can be generated using the digest function. Any changes to the message will (ideally) result in a different hash. An attacker must not only replace the message but also the digest with a new valid pair. A Message Authentication Code (MAC) algorithm takes two inputs, a message and a secret key, and produces a MAC that allows you to verify the integrity and authenticity of the message: any changes to the message or the secret key will (ideally) result in a different MAC. No one without access to the key should be able to generate a MAC calculation that verifies this. The HMAC function combines the MAC with the result of a hash function. It is a way to expand the use of hash functions since the result does not depend only on the hash of the message but on another parameter, which can be the user's secret key.
 - (b) Calculate the checksum of the encrypted file using 'openssl dgst' and sign it with the private key using the sign function to send it to the destination Fog node. For algorithms that do not support signing, calculate the HMAC over the encrypted file with a password to authenticate the message.
 - (c) Repeat the command to obtain metrics using the tools *ps*, *perf*, and *strace*.
 - (d) Extract the selected metrics and save them in a file.
- (5) Decryption
- (a) Verify the integrity of the file with the public key. For algorithms that do not support signing, calculate HMAC over the encrypted file and compare it with the received one.
 - (b) Decrypt using the same symmetric algorithm and password used for encryption with the 'openssl enc -d' function.
 - (c) Repeat decryption to obtain metrics using the tools *ps*, *perf*, and *strace*.
 - (d) Extract the selected metrics and save them in a file.

2.4. Datasets

Drawing from the literature and the research uncovered during the formulation of the theoretical framework, it became evident that the majority of tests comparing different algorithms primarily utilized plain text files smaller than 10 MB. This trend is exemplified by Patel and Bhavsar (2021), who employed a .csv file containing patient medical information. Notably, a limited number of studies opted for unstructured datasets, as demonstrated by Vishwanath et al. (2016), who utilized a dataset comprising 10 MB images, characters, numbers, and pictorial representations.

In the context of this study, three distinct types of data were chosen for testing: text, images, and video. Considering that in the Internet of Things (IoT), the data exchanged between nodes tends to be relatively small, files exceeding 100 MB were excluded from the comparative analysis.

The selected files are delineated in Table 2. The plain text data is formatted as .csv and originates from databases extracted from IoT sensors available on the Internet. In the case of images, three formats (.jpg, .bmp, and .png) were employed, featuring photographs captured by IP cameras. As for videos, in .mp4 format, they correspond to footage recorded by drones.

Table 2. Dataset information.

Type	Name	Size in MBytes	Source
CSV Text	transport	1.1	Carpineti et al. (n.d.)
	IoT-telemetry1	50	Kaggle (n.d.)
	home	100	Bouzahzah (n.d.)
MP4 Video	sheeps_70858	1.1	Pixabay (n.d.)
	cropfield	50	Ritter, P.T. (n.d.)
	sunflowers	100	Pixabay (n.d.)
Image	temp_nubes2	1.1	INPE, D (n.d.)
	shelter_ingreso_50	50	Martinez (n.d.)
	peaje_100	100	Martinez (n.d.)

2.5. Metrics

Considering that the Fogs nodes are limited in resources (storage, computing, memory and energy), the following metrics were taken into account for the tests to be carried out:

- CPU Consumption: Measures the percentage of CPU used with the *perf* tool.
- IOWait: Measures the percentage of time that the CPU remains in the IOWait state (waiting for the completion of Input/Output operations).
- Memory Usage: measures the percentage of RAM memory usage with the *perf* and *ps* tool.
- External Memory Access: Counts the number of external memory accesses using the *perf* tool.
- Execution time: Measured in seconds with the *time* command.
- Power consumption: A USB consumption meter is used to determine the consumption of the Fog node during testing. The meter used is from the KEWEISI brand which allows measuring the voltage level in Volts and the current consumption in Amperes. Before starting the measurements, the consumption in the rest state of the RPI and without any sensor connected was determined. For availability reasons, this metric could only be measured on RPI 3, yielding a consumption of 250 mA in the idle state.

Table 3 describes the commands used to measure metrics, where the first column indicates the metric and second column shows the command and the arguments used. The metrics were assessed using the previously mentioned datasets. In the context of decryption, measurements were conducted on the encrypted files, resulting in a larger file size

Table 3. Software tools used to measure.

Metric	Software command
CPU Consumption	<i>perf_5.10 stat</i>
IOWait	<i>iostat</i>
Memory usage	<i>ps -axo pid,pmem,cmd</i>
External memory access	<i>perf_5.10 stat -e ext_mem_req, L1-dcache-loads, cache-misses, cache-references</i>
Execution time	<i>perf_5.10 stat</i>

compared to the original. However, for a more accurate representation of the results, the data was tabulated based on the original file size. To ensure statistical robustness, each test was executed five times, and the average of each metric was computed. The outcomes of each test were stored in a csv (comma-separated values) format output file by each script, following this pattern:

```
File, symmetric algorithm, data type, size, test, repeat #,  
checksum algorithm, time in seconds, % CPU used, % memory,  
%external memory, %read calls, read time (sec),  
%calls writing time, writing time (sec),  
time spent on system calls (sec).
```

3. Results and discussion

Due to the large number of tests carried out, the most notable results are summarized and discussed below. All results are available in a [git front repository](#) (Olivera, 2024). As described in Section 2.2, we use three versions of Raspberry Pi: RPi 1 model b+, RPi 3 Model B and RPi 4.

3.1. Node 1: RPi 3

Based on the analysis of the tests carried out in Node 1 (RPi 3), the results obtained can be summarized as follows. For encryption and decryption functions, in terms of memory usage percentage, there are no significant differences observed among the different algorithms. When accessing external memory, AES demonstrates lower memory requirements, particularly noticeable for larger files (50 MB and 100 MB, see Table 4). Conversely, for smaller files, DES3 incurs fewer memory accesses. Chacha20 stands out as the least CPU-intensive and fastest algorithm.

Regarding the checksum function, the results show no discernible relationships or patterns. The obtained values exhibit similarity and fall within the same order. While the tables below highlight algorithms that slightly distinguish themselves, an overarching pattern cannot be established based on file type or size.

Regarding the IOWait metric, based on the results obtained using all possible combinations of algorithms, it can be established that, across the three datasets, as the file size increases, the CPU usage percentage decreases for encryption and decryption tasks. This

Table 4. Comparison table of the ‘external memory access’ counter measured with perf. The values are expressed in thousands.

Algorithm	Function	Image			Text			Video		
		1	50	100	1	50	100	1	50	100
aes-256-cbc	decryption	733	9015	17,939	738	8863	17,226	745	9012	17,097
	encryption	681	6901	13,104	686	6769	13,521	693	6758	13,073
	checksum	149	1285	2585	146	1327	2599	148	1328	2546
chacha20	decryption	750	9708	19,173	759	9608	19,022	758	9767	19,414
	encryption	704	7516	14,269	700	7373	14,096	707	7168	14,147
	checksum	150	1300	2502	144	1261	2633	147	1312	2635
des3	decryption	600	13,671	27,202	604	13,365	26,394	611	13,353	26,690
	encryption	530	9860	19,429	536	9884	20,117	1	10,063	19,830
	checksum	147	1368	2464	148	1278	2532	147	1358	2438

is due to the increased time the CPU spends in the IOWait state, i.e. idle, waiting to perform an IO operation (reading from or writing to the storage device). However, no clear relationship can be established between the checksum calculation and the data type or size. In general, it can be observed that the encryption function requires the least CPU usage, while the checksum calculation consumes the most. Additionally, the Chacha20 algorithm utilizes a lower percentage of the total CPU, and its combination with ECDSA and SHA3-512 in the encryption function achieves the minimum usage percentage. On the other hand, the maximum CPU usage percentage ranges between 90.7% and 90.9% for checksum calculation on 100 MB files. However, there is no significant decrease in CPU consumption observed for smaller file sizes.

3.2. Node 2: RPi 1

Given that the selected device as Node 2 is an RPi 1, manufactured in 2012 with considerably lower resources than the RPi 3 (manufactured in 2016) and the RPi 4 (from 2019) (see Section 2.2), nearly unavailable in the market and seldom used in new development projects, it was deemed necessary to conduct tests only on one symmetric algorithm and one checksum algorithm for comparative purposes. Based on the analysis of the Node 1 results, the Chacha20/SHA3-512 combination, which exhibited the best performance, was chosen. The main results obtained on Node 2 are summarized below.

3.2.1. Memory usage percentage

As can be seen in Figure 1, regarding this metric, no significant differences among the different algorithms were observed, and no relationship between data type, file size, and used algorithms could be established based on the results. Compared to the RPi 3, the memory usage percentages are 2 to 5 times higher than those recorded in Node 1. However, it is essential to note that the RPi 1 has half the RAM of the RPi 3.

3.2.2. CPU usage percentage

Relationships or patterns in the results cannot be established, but it is noteworthy that the checksum function is the most CPU-intensive function, regardless of the file type and size, followed by decryption. The use of the *sar* tool revealed that the CPU is continuously in use during the execution of this function, unlike encryption and decryption, which must wait for input/output data. In most results, the consumption is approximately half of what the RPi 3 requires, it can be explained based on the following runtime metric.



Figure 1. Metrics comparison between RPi 1 and RPi 3 for ECDSA/Chacha20/SHA3-512.

3.2.3. Execution time

As the file size increases, the execution time of each function increases, but it is not influenced by the type of data in the file. Based on the results, it can also be determined that encryption is the fastest function to execute for 50 and 100 MB files, regardless of the algorithm used, while the checksum is the fastest for 1 MB files. In contrast to the RPi 3, the execution times increase between 3 and 10 times, even reaching 20 times slower in some specific cases. For example, It can consider the decryption of a 100 MB video file and apply the combination of algorithms ECDSA/Chacha20/Sha3-512 to compare the metrics.

3.3. Node 3: RPi 4

In this node, it is executed all five repetitions of the scripts for different algorithms and files. Below is a synthesis of the main results.

3.3.1. Encryption and decryption functions

In terms of memory usage percentage, there are no significant differences among the various algorithms. Encryption is the most memory-intensive function, but it differs only slightly from decryption. Similar to Node 1, ECC requires more memory, while DH is the algorithm that utilizes the least percentage.

Concerning external memory accesses, larger file sizes result in more accesses, with Chacha20 performing the best in this metric. Conversely, DES3 requires the highest number of accesses to external memory. Just like in Node 1, Chacha20 is the algorithm that consumes the least CPU resources and is the fastest.

3.3.2. Checksum function

This function requires the least memory and external memory access, and it is also the fastest to execute. However, it consumes the highest percentage of CPU. The explanation for this result is assumed to be that, similar to Nodes 1 and 2, the CPU spends less time in the IDLE or IOWait state than in other functions. Regarding algorithms, SHA-1 consumes the least memory and CPU, while SHA3-512 is the slowest.

3.4. Summary of results for nodes 1, 2, and 3

Based on the tests conducted in this work, an analysis of the results obtained from encryption and decryption functions on one hand, and checksum functions on the other can be separately discussed.

3.4.1. Node 1 (RPI 3)

No significant differences were observed in memory usage percentage among different asymmetric and symmetric algorithms. Slight increases in consumption were noted with ECC, while DH exhibited a more conservative use of resources. For encryption and decryption functions, AES required less external memory for larger files (50 MB and 100 MB), while DES3 performed better for smaller files considering memory accesses. Chacha20 emerged as the algorithm with the least CPU resource consumption and the fastest execution. Checksum functions showed no distinct patterns among algorithms; values were similar across all algorithms.

3.4.2. Node 2 (RPI 1)

Only asymmetric algorithms combined with Chacha20 as the symmetric algorithm and Sha3-512 as the checksum algorithm were evaluated. Similar to Node 1, there were no significant differences in memory usage among different algorithms. External memory accesses could not be tracked on RPI 1. CPU consumption and execution time did not exhibit significant differences among algorithms, with slight variations in execution time for 1 MB files.

3.4.3. Node 3 (RPI 4)

No significant differences in memory usage percentage were observed among different algorithms. Similar to Node 1, encryption was the most memory-consuming function, with ECC requiring more memory, and DH utilizing the least percentage. Chacha20 excelled in external memory accesses for larger file sizes, while DES3 required more accesses. Chacha20 remained the algorithm with the least CPU consumption and the fastest execution. Checksum function exhibited similar characteristics as in Node 1, with SHA1 consuming the least memory and CPU, and SHA3-512 being the slowest.

Analysing all possible algorithm combinations across the three nodes, it can be concluded that none of the algorithms consume a significant percentage of memory that could affect other tasks on the nodes. There is no discernible relationship between the type and size of data. Regarding external memory accesses, they increase with larger files, with decryption requiring the most accesses, followed by encryption and checksum. CPU usage decreases as file size increases for encryption and decryption, due to increased time in the IOWait state. Checksum function did not exhibit a clear relationship with data type or size. Lastly, the execution time increases with file size, but it is independent of data type.

Regarding energy consumption, measurements were taken only in Node 1 using Elliptic Curve and RSA algorithms for a 100 MB video file. Chacha20 and AES-256-CBC were selected as symmetric algorithms due to their faster performance for 100 MB files. Chacha20 showed lower consumption for both, encryption and decryption compared to AES-256-CBC, with a peak consumption of 0.5 A in decryption for all combinations.

3.5. Trade-off between security and performance

Based in the experimental results, different analysis can be realized in order to evaluate the trade-off between security and performance.

3.5.1. Memory usage and performance

In analysing encryption and decryption functions across the different Raspberry Pi nodes (RPI 1, RPI 3, and RPI 4), it is clear that there are no significant differences in memory usage percentages among the various algorithms. However, encryption consistently proves to be the most memory-intensive operation, slightly more so than decryption. Notably, ECC (Elliptic Curve Cryptography) requires more memory than other algorithms, whereas DH (Diffie-Hellman) uses the least. A key observation is that Chacha20 consistently performs well across multiple metrics, requiring the fewest external memory accesses and the least CPU resources, making it the most efficient algorithm for large file sizes. On the other hand, DES3 requires the highest number of external memory accesses, highlighting a performance trade-off when using DES3 for security. High security (ECC, DES3) comes with increased memory and external memory access costs, which

might reduce the overall system performance. Optimal performance is achieved using Chacha20, which balances low memory use and minimal CPU consumption, ideal for lightweight IoT edge devices like Raspberry Pis.

The checksum function uses the least memory and external memory accesses but consumes the highest percentage of CPU, making it a clear candidate for scenarios where memory is scarce, but CPU utilization can be sacrificed.

3.5.2. CPU usage and execution time

Regarding CPU usage, the checksum function stands out as the most CPU-intensive, consuming nearly 90% of the CPU when applied to large file sizes (e.g. 100 MB). This is mainly because the CPU spends less time in IDLE or IOWait states, unlike encryption and decryption, which are more dependent on I/O operations. SHA-1 proves to be the least demanding on both memory and CPU, while SHA3-512 is the slowest algorithm.

Interestingly, as file sizes increase, the CPU usage percentage decreases for encryption and decryption tasks. This is attributed to increased time in the IOWait state, meaning that the CPU is often waiting for I/O operations rather than performing encryption or decryption computations. Encryption emerges as the fastest function to execute for larger files (50 MB, 100 MB), regardless of the algorithm used.

While checksum functions can provide robust data integrity checks, they demand significant CPU resources, which could slow down real-time processing in Fog nodes. Chacha20/SHA3-512 consistently achieves the lowest CPU usage for encryption, making it suitable for environments where energy efficiency and CPU conservation are critical.

3.5.3. Node-specific considerations

Node 1 (RPI 3) shows similar patterns in memory usage and CPU consumption across different algorithms, with Chacha20 continuing to perform best in terms of CPU and memory usage. The encryption function generally consumes the least CPU, with Chacha20 combined with ECDSA and SHA3-512 resulting in minimal CPU usage. Node 2 (RPI 1), with its significantly lower resources, presented a challenge. However, even here, the combination of Chacha20/SHA3-512 showed favourable results. Given that the RPi 1 has half the RAM of RPi 3, memory usage percentages are notably higher, between 2 to 5 times more than those recorded in Node 1. CPU usage remains high for checksum functions, while encryption retains the lowest CPU consumption. Node 3 (RPI 4) follows a similar pattern to Node 1, where Chacha20 is the least demanding algorithm, and SHA1 is the least CPU-intensive. The combination of Chacha20, ECDSA, and SHA3-512 continues to minimize CPU and memory utilization.

RPi 1 is less capable of handling heavy encryption workloads, and using lighter algorithms such as Chacha20 is critical to achieving a balance between security and performance. RPi 3 and RPi 4 can handle more complex algorithms (ECC, RSA) but with increased memory and CPU costs.

3.5.4. Energy efficiency

Energy consumption measurements taken from Node 1 using Chacha20 and AES-256-CBC show that Chacha20 consumes less energy during encryption and decryption compared to AES-256-CBC. The peak energy consumption during decryption for all combinations was 0.5A.

Chacha20 emerges as the best candidate for energy efficiency, ideal for use cases where battery life or low power consumption is essential, such as in IoT or remote deployments.

4. Conclusions

This study has aimed to conduct an analysis and comparison of encryption and message verification algorithms within a Fog Computing Architecture. The goal was to identify an optimal combination that efficiently utilizes Fog node resources, avoids communication delays, and ensures data security. Several algorithms known for their speed and resource efficiency based on literature reviews were selected.

Subsequently, three models of Raspberry Pi were employed for real hardware testing, representing an environment close to what could be utilized in a Fog Computing setting. To facilitate the testing, scripts were developed for the repetition and variation of parameters, assessing critical metrics for Fog nodes, including memory consumption, CPU usage, and energy consumption. Results were analysed and compared to determine suitable algorithms for each dataset.

In summary, Chacha20 is consistently the best performer in terms of memory usage, CPU consumption, and energy efficiency, making it the optimal choice for Fog computing environments, particularly in Raspberry Pi setups. ECC and SHA3-512 provide enhanced security but come with higher CPU and memory usage, making them suitable for applications where security is prioritized over performance. For resource-constrained nodes like RPi 1, it is critical to use lightweight algorithms such as Chacha20 to maintain an acceptable performance level while still ensuring basic security.

The outcomes allowed us to analyse the efficiency of algorithms in the Fog Computing environment. While a single standout combination applicable in all situations wasn't identified, it was determined that the data type being processed does not significantly influence algorithm selection. This insight suggests the feasibility of applying security algorithms in a variety of IoT systems within Fog Computing platforms. Additionally, the increase in data size doesn't proportionally impact all metrics, and some metrics show minimal influence. Regarding the resources consumed by encryption and verification tasks, it is evident that adding security to data compromises CPU usage and demands time, which may be critical in real-time applications but less so in sporadic IoT sensor transmissions.

Acknowledgments

We thank Pablo Godoy for lending us a Raspberry Pi 4 unit to carry out some of the experiments in this work.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by Universidad Nacional de Cuyo projects number [SIIP T001-T4 2022/2024] and [06/B052-T1], and by Agencia Nacional de Promoción de la Investigación, el Desarrollo

Tecnológico y la Innovación grant code [PICT-2020-SERIEA-00743] and (Consejo Nacional de Investigaciones Científicas y Técnicas).

ORCID

Pablo Javier Vidal  <http://orcid.org/0000-0001-6502-8010>

Ana Carolina Olivera  <http://orcid.org/0000-0001-7825-1959>

Emmanuel N. Millán  <http://orcid.org/0000-0002-5666-8355>

References

- Abood, O. G., & Guirguis, S. K. (2018). A survey on cryptography algorithms. *International Journal of Scientific and Research Publications*, 8(7), 495–516. <https://doi.org/10.29322/IJSRP.8.7.2018.p7978>
- Alshambri, H. A., & Alassery, F. (2021). Securing Fog Computing for e-learning system using integration of two encryption algorithms. *Journal of Cyber Security*, 3(3), 149–166. <https://doi.org/10.32604/jcs.2021.022112>
- Bernstein, D. J. (2006). Curve25519: New diffie-hellman speed records. In *Public Key Cryptography-PKC 2006* (pp. 207–228). Berlin: Springer.
- Bernstein, D. J., ECRYPT II, T. (2008). Chacha, a variant of salsa20. In *Workshop Record of SASC 2008*.
- Bertoni, G., Daemen, J., Peeters, M., & Van Assche, G. (2013). The Keccak reference. Keccak Team.
- Bouzahzah, O. (n.d.). Smart home dataset. <https://www.kaggle.com/code/offmann/smart-home-dataset>.
- Carpinetti, C., Lomonaco, V., Bedogni, L., Di Felice, M., & Bononi, L. (n.d.). Transportation mode detection dataset. <https://www.kaggle.com/datasets/fschwartz/tmd-dataset-5-seconds-sliding-window>.
- Cheikhrouhou, O., Mershad, K., Jamil, F., Mahmud, R., Koubaa, A., & S. R. Moosavi (2023). A lightweight blockchain and Fog-enabled secure remote patient monitoring system. *Internet of Things*, 22, Article 100691. <https://doi.org/10.1016/j.iot.2023.100691>
- Chi, L., & Zhu, X. (2017). Hashing techniques: A survey and taxonomy. *ACM Computing Surveys (Csur)*, 50(1), 1–36. <https://doi.org/10.1145/3047307>
- De Giusti, A. E., Naiouf, M., Medina, S., Antueno, J. d., De Giusti, L. C., Lanciotti, J., Tinetti, F. G., Chichizola, F., Rucci, E., Romero, F., Sanz Pousa, A., Encinas, D., Paniego, J. M., Pi Puig, M., Libutti, L. A., & Costanzo, M. (2021). Arquitecturas Edge-Fog-Cloud en procesamiento distribuido. In *XXIII Workshop de Investigadores en Ciencias de la Computación (WICC 2021, Chilecito, La Rioja)*.
- Dobbertin, H. (1998). Cryptanalysis of MD4. *Journal of Cryptology*, 11(4), 253–271. <https://doi.org/10.1007/s001459900047>
- Dworkin, M. (2001). The aes algorithm. *NIST FIPS*, 197, 1–46.
- Dworkin, M. (2015). Secure hash standard (SHS). *NIST FIPS*, 180(4), 1–36.
- Froiz-Míguez, I., Fernández-Caramés, T. M., Fraga-Lamas, P., & Castedo, L. (2018). Design, implementation and practical evaluation of an iot home automation system for Fog Computing applications based on mqtt and zigbee-wifi sensor nodes. *Sensors*, 18(8), 2660–2702.
- INPE, D. (n.d.). Satellite temperature dataset. <https://www.cptec.inpe.br/dsat/>.
- Janardhana-Naidu, J., & Ganesh, E. (2023, February). A novel blockchain-based lightweight encryption technique in Fog based iot for personal healthcare data application. *International Journal of Intelligent Systems and Applications*, 11, 119–128.
- Johnson, D., Menezes, A., & Vanstone, S. (2001). Elliptic curve digital signature algorithm (ecdsa). *International Journal of Information Security*, 1(1), 36–63. <https://doi.org/10.1007/s102070100002>
- Kaggle (n.d.). IoT telemetry dataset. <https://www.kaggle.com/datasets/ajmal00147/iot-telemetry-data>.
- Kahvazadeh, S. (2019). Security architecture for Fog-to-Cloud continuum system.
- Khan, G., Gola, K. K., Kanauzia, R., & Kumar, S. (2022). Secure architecture to support iot based on Fog Computing. *Procedia Computer Science*, 215, 608–617. <https://doi.org/10.1016/j.procs.2022.12.063>
- 4th International Conference on Innovative Data Communication Technology and Application.

- Martínez, L. (n.d.). Images of IP cameras dataset. <https://www.kaggle.com/datasets/laylamartinez/camaras-ip>.
- Mitali, V. K., & Sharma, A. (2014). A survey on various cryptography techniques. *International Journal of Emerging Trends & Technology in Computer Science (IJETTCS)*, 3(4), 307–312.
- The National Institute of Standards and Technology (NIST) (2017). Triple data encryption algorithm (TDEA) specification. NIST Special Publication (800-67).
- Olivera, A. C. (2024). Git repository with results dataset. Online; Retrieved June 14, 2024, from <https://gitfront.io/r/acolivera/RNH4yiELBN2X/Martinez-TJIT-2024/>.
- Patel, V., & Bhavsar, S. (2021). Development of security platform in Fog Computing. *GIT-Journal of Engineering and Technology*, 13, 57–63.
- Perera, C., Qin, Y., Estrella, J. C., Reiff-Marganiec, S., & Vasilakos, A. V. (2017). Fog Computing for sustainable smart cities: A survey. *ACM Computing Surveys (CSUR)*, 50(3), 1–43. <https://doi.org/10.1145/3057266>
- Pixabay (n.d.). Drone sheep video dataset. <https://pixabay.com/es/videos/oveja-reba>.
- Pixabay (n.d.). Drone sunflowers video dataset. <https://pixabay.com/es/videos/girasoles-abejas-campo-bloom-59483>.
- Qin, M., Li, M., & Othman Yahya, R. (2023). Dynamic IoT service placement based on shared parallel architecture in Fog-cloud computing. *Internet of Things*, 23, Article 100856. <https://doi.org/10.1016/j.iot.2023.100856>
- Ramzan Ali, A. B., Alagumariappan, P., Vijayalakshmi, S., Gughan, N., & Bhaskar, K. (2020). Analysis of secure transfer of healthcare data using Fog Computing. In *2020 International Conference on Smart Electronics and Communication (ICOSEC)* (pp. 529–531). IEEE.
- Raspberry Pi (n.d.). Official website. <https://www.raspberrypi.com/>.
- Ritter, P.T. (n.d.). Drone cropfields video dataset. <https://pixabay.com/es/videos/cosecha-cosecha-de-grano-agricultura-86119/>.
- Rivest, R. L., Shamir, A., & Adleman, L. M. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120–126. <https://doi.org/10.1145/359340.359342>
- Shen, Y., Sun, Z., & Zhou, T. (2021). Survey on asymmetric cryptography algorithms. In *2021 International Conference on Electronic Information Engineering and Computer Science (EIECS)* (pp. 464–469). IEEE.
- Shruti, , Rani, S., & Srivastava, G. (2024). Secure hierarchical Fog Computing-based architecture for industry 5.0 using an attribute-based encryption scheme. *Expert Systems with Applications*, 235, Article 121180. <https://doi.org/10.1016/j.eswa.2023.121180>
- Vishwanath, A., Peruri, R., & He, J. S. (2016). Security in Fog Computing through encryption. DigitalCommons@ Kennesaw State University Kennesaw, Georgia, USA.
- Wang, X., & Yu, H. (2005). How to break MD5 and other hash functions. In *International Conference on Advances in Cryptology-CRYPTO 2005* (pp. 19–35). Springer.
- Weber, R. H. (2015). Internet of things: Privacy issues revisited. *Computer Law & Security Review*, 31(5), 618–627. <https://doi.org/10.1016/j.clsr.2015.07.002>
- Yi, S., Hao, Z., Qin, Z., & Li, Q. (2015). Fog Computing: Platform and applications. In *2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb)* (pp. 73–78). IEEE.
- Zhao, C. W., Jegatheesan, J., & Loon, S. C. (2015). Exploring iot application using raspberry pi. *International Journal of Computer Networks and Applications*, 2(1), 27–34.